

Information Leakage: Differential Attacks

2025

1 Introduction

This handout explains differential side channel attacks in considerable detail, with a focus on power attacks on implementations of the Advanced Encryption Standard (AES) [Nat23].

AES is a block cipher. This means that it takes data of a specific (block) length as input, and using a secret key, produces a ciphertext block. Used “just as is” it provides computational one-way security, therefore it is typically used as a building block as part of more complex primitives. The high-level flow of AES encryption is illustrated in Alg. A. Notice that this description suggests that the round keys are generated prior to the rounds: in practice, when there is not enough memory/storage available, this tends to be interleaved with the encryption round.

Alg. A: AES Encryption

Provided an input m and key k , this delivers an AES encryption c of the input as a result.

```
input:  $m, k$ 
output:  $c$ 
state =  $m$ 
rk = KeyExpansion( $k$ )
AddRoundKey(state,  $rk_0$ )
for  $r = 1$  step 1 to 9
    SubBytes(state)
    ShiftRows(state)
    MixColumns(state)
    AddRoundKey(state,  $rk_r$ )
end
SubBytes(state)
ShiftRows(state)
AddRoundKey(state,  $rk_{10}$ )
 $c = \text{state}$ ;
```

1.1 Visual inspection of a trace

Every algorithm that runs on a cryptographic device is executed in a sequential manner. Even in hardware, cryptographic (round) functions are too complex to be executed in a single step. The mathematical description of an algorithm has first to be translated into some (high-level) language. This description is eventually translated (by a compiler) into instructions that are supported by a device.

In our concrete example, AES, there are ten rounds. Each round consists of four round transformations, which are called AddRoundKey, SubBytes, ShiftRows, and MixColumns, each of which takes in one or several bytes of the state. Suppose that AES is implemented in software on a processor that is based on an ARM architecture, the round functions will be translated into the instructions of the ARM instruction set that are available on the processor. The instruction set of most processors consists of arithmetic instructions (such as addition), logical instructions (such as exclusive-or), data transfer instructions (such as move), and branching instructions (such as jump).

Each instruction works on a number of bytes and utilises different components of the processor, such as the arithmetic-logic unit, memory (external or internal RAM or ROM), or some peripheral (such as a communication port). These individual components of the processor differ in functionality and implementation. Therefore, they have a characteristic power consumption, which leads to a clear pattern in the power trace. The possibility to distinguish instructions within a trace can lead to a serious security problem if the sequence of instructions directly depends on the key (as we have seen in the case of RSA).

Let's perform a visual inspection of a power trace of an AES implementation on a simple microcontroller. In that AES implementation there are no data dependencies in the instruction flow, thus a visual inspection will not give away the secret key. Figure 1 shows a power trace of a full AES encryption run on the simple microcontroller. The 9 full rounds are clearly visible, as well as the last round in which MixColumns is skipped.

As a next step we zoom into the first two rounds, and then a single round of this trace, and finally into a short sequence of low level instructions. Figure 2 zooms in on the first two rounds. The first round is located between 0.06 ms and 0.73 ms, the second round is located between 0.84 ms and 1.52 ms. If you struggle locating the rounds, then look out for three tiny peaks in the upper half of the picture. Figure 3 shows just a single round where we have “compressed” the trace (by identifying each clock cycle and replacing it with a single representative value). The round starts at clock cycle 555 and ends at clock cycle 4305. There are three different patterns visible in this picture. Many typical AES implementations in software first perform the round functions that operate on

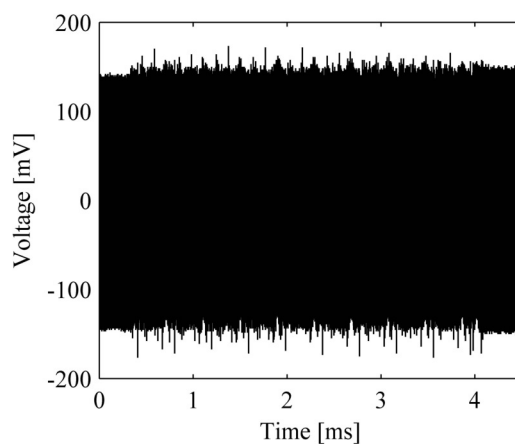


Figure 1: Full AES encryption.

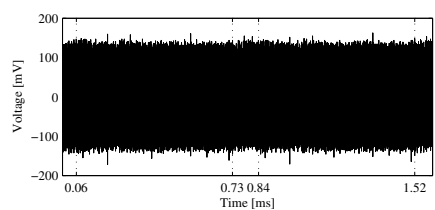


Figure 2: Two rounds of AES (no compression)

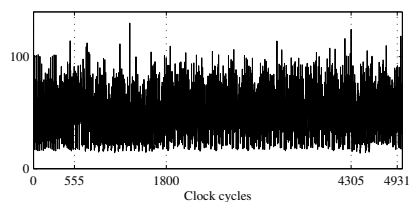


Figure 3: One round of AES (with compression)

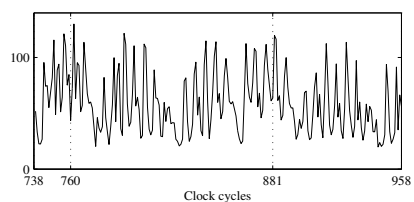


Figure 4: AddRoundKey, Sub-Bytes, and ShiftRows(twice).

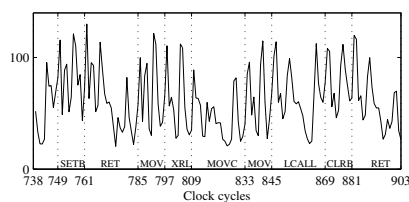


Figure 5: AddRoundKey, Sub-Bytes, and ShiftRows with annotation.

```

LCALL SET_ROUND_TRIGGER
MOV A,ASM_input + 0      ; load a0
XRL A,ASM_key + 0        ; add k0
MOVC A,@A + DPTR         ; S-box look-up
MOV ASM_input, A         ; store a0
LCALL CLEAR_ROUND_TRIGGER

```

Figure 6: The sequence of assembly instructions that corresponds to Figure 5.

individual bytes and then perform MixColumns, which operates on a column (4 bytes) of the AES state. Your implementation might follow the same principle, or you may have chosen to process the state column by column (thus interleaving a single column of MixColumns with the other round functions).

If the key generation is done on-the-fly, it will have to occur in between the rounds so the next round key is ready. The implementation of AES that we are examining does just that. Therefore, the first pattern in Figure 3 corresponds to the sequence of AddRoundKey, SubBytes, and ShiftRows operations, the second pattern corresponds to MixColumns and the third pattern corresponds to the generation of the round key.

Within the first pattern, one can count 16 tiny peaks (on the bottom of the trace) that are closely located next to each other. Each peak corresponds to a sequence of AddRoundKey, SubBytes, and ShiftRows. In Figure 4, we zoom in on Figure 3 and show the power consumption of the first two peaks only. The first peak is located between clock cycle 760 and 881. The shape of the trace shortly after clock cycle 760 is very similar to the shape of the trace after clock cycle 881. To help interpreting this trace, I have added the sequence of assembly instructions in Figure 6.

First, the function `SET_ROUND_TRIGGER` is called to set a trigger signal. The `SET_ROUND_TRIGGER` function executes a `SETB` (set bit) and a `RET` (return) instruction. Then one byte of the AES state is processed. Afterwards, the trigger signal is cleared by calling the `CLEAR_ROUND_TRIGGER` function, which executes a `CLRB` (clear bit) and a `RET` instruction. On this microcontroller, different instructions require different numbers of clock cycles for their execution. `SETB`, `MOV`, and `XRL` require 12 clock cycles whereas the other instructions require 24 clock cycles. Figure 5 shows an annotated sequence of instructions corresponding to the assembly.

2 Differential power analysis of AES

Standard DPA attacks exploit the data dependent component of the power consumption. Such attacks often use a large number of power traces and analyse the power consumption at a fixed, single moment of time as a function of the processed data.

The key characteristic, and the reason behind the term “differential”, is that we assume that we can observe varying inputs to the implementation under attack (we also assume that the key is kept constant)¹.

¹The term “differential cryptanalysis” was coined to refer to a classical cryptanalytic method, which varies the inputs in specific ways and then looks how the difference in inputs manifests as difference in

There exists a general attack strategy that is used by all DPA style attacks. This strategy consists of five steps, which I am now taking directly from [MOP07]. There exists a general attack strategy that is used by all DPA style attacks. This strategy consists of five steps, which I am now taking directly from [MOP07].

Step 1: Choosing an Intermediate Result of the Executed Algorithm The first step of a DPA attack is to choose an attack target: this must be some intermediate result of the cryptographic algorithm that is executed by the attacked device. The chosen intermediate result (or target function, or selection function) needs to be a function of some known data d and a small part k of the key.

Step 2: Measuring the Power Consumption The second step of a DPA attack is to measure the power consumption of the cryptographic device while it encrypts or decrypts D different messages (these maybe plaintexts or ciphertexts). For each of these encryption or decryption runs, the attacker needs to know the corresponding data value d that is involved in the calculation of the intermediate result chosen in step 1. We write these known data values as vector $\mathbf{d} = (d_1, \dots, d_D)'$, where d_i denotes the data value in the i^{th} encryption or decryption run.

During each of these runs the attacker records a power trace. We refer to the power trace that corresponds to data block d_i as $\mathbf{t}'_i = (t_{i,1}, \dots, t_{i,T})$, where T denotes the length of the trace. The attacker measures a trace for each of the D data blocks, and hence, the traces can be written as matrix $\mathbf{T}$ of size $D \times T$. It is important for DPA attacks that the measured traces are correctly aligned. This means that the power consumption values of each column $\mathbf{t}_j$ of the matrix $\mathbf{T}$ need to be caused by the same operation. In practice this may not be the case and then one has to process power traces carefully. There are a number of alignment and general signal processing techniques out there, but they are out of scope for us.

Step 3: Predicting Intermediate Values The next step of the attack is to calculate a *hypothetical intermediate value* for every possible choice of k . We write these possible choices as vector $\mathbf{k} = (k_1, \dots, k_K)$, where K denotes the total number of possible choices for k . In the context of DPA attacks, we usually refer to the elements of this vector as key hypotheses. Given the data vector $\mathbf{d}$ and the key hypotheses $\mathbf{k}$, an attacker can easily calculate hypothetical intermediate values $f(d, k)$ for all D encryption runs and for all K key hypotheses. This calculation (1) results in a matrix $\mathbf{V}$ of size $D \times K$. The

outputs. The cost of this kind of attack is significant for modern ciphers like AES, and it was out of frustration about this cost that P. Kocher came up with the idea of using **additional** information as part of such an attack strategy.

first part of Figure 7 illustrates this calculation step.

$$v_{i,j} = f(d_i, k_j) \quad i = 1, \dots, D \quad j = 1, \dots, K \quad (1)$$

Column j of $\mathbf{V}$ contains the intermediate results that have been calculated based on the key hypothesis k_j . It is clear that one column of $\mathbf{V}$ contains those intermediate values that have been calculated in the device during the D encryption or decryption runs. Remember, $\mathbf{k}$ contains all possible choices for k . Hence, the value that is used in the device is an element of $\mathbf{k}$. We refer to the index of this element as ck . Hence, k_{ck} refers to the key of the device. The goal of DPA attacks is to find out which column of $\mathbf{V}$ has been processed during the D encryption or decryption runs. As soon as we know which column of $\mathbf{V}$ has been processed in the attacked device, we immediately also know k_{ck} .

Step 4: Mapping Intermediate Values to Power Consumption Values The next step of a DPA attack is to map the hypothetical intermediate values $\mathbf{V}$ to a matrix $\mathbf{H}$ of *hypothetical power consumption values*, see Figure 7. For this purpose, the adversary either relies on a standard leakage model (such as the Hamming weight), or they determine a more accurate model via a process called profiling (templating).

Step 5: Comparing the Hypothetical Power Consumption Values with the Power Traces After having mapped $\mathbf{V}$ to $\mathbf{H}$, the final step of a DPA attack can be performed. In this step, each column $\mathbf{h}_i$ of the matrix $\mathbf{H}$ is compared with each column $\mathbf{t}_j$ of the matrix $\mathbf{T}$. This means that the attacker compares the hypothetical power consumption values of each key hypothesis with the recorded traces at every position. The result of this comparison is a matrix $\mathbf{R}$ of size $K \times T$, where each element $r_{i,j}$ contains the result of the “comparison” between the columns $\mathbf{h}_i$ and $\mathbf{t}_j$. The comparison is done based on a statistical distinguisher. In practice the correlation coefficient tends to be an excellent choice for this purpose. The idea is then that a higher value $r_{i,j}$ indicates a better match between the columns $\mathbf{h}_i$ and $\mathbf{t}_j$. The key of the attacked device can hence be revealed by checking which key hypothesis leads to the best (highest) match. The indices of the highest values of the matrix $\mathbf{R}$ also reveal the positions at which the chosen intermediate result has been processed and the key that is used by the device.

If all values of $\mathbf{R}$ are approximately the same, this indicates the attack hasn’t succeeded. Reasons for failure include an insufficient amount of traces, a bad choice for an intermediate value, or an inappropriate power model. Or, there simply is a bug in your code.

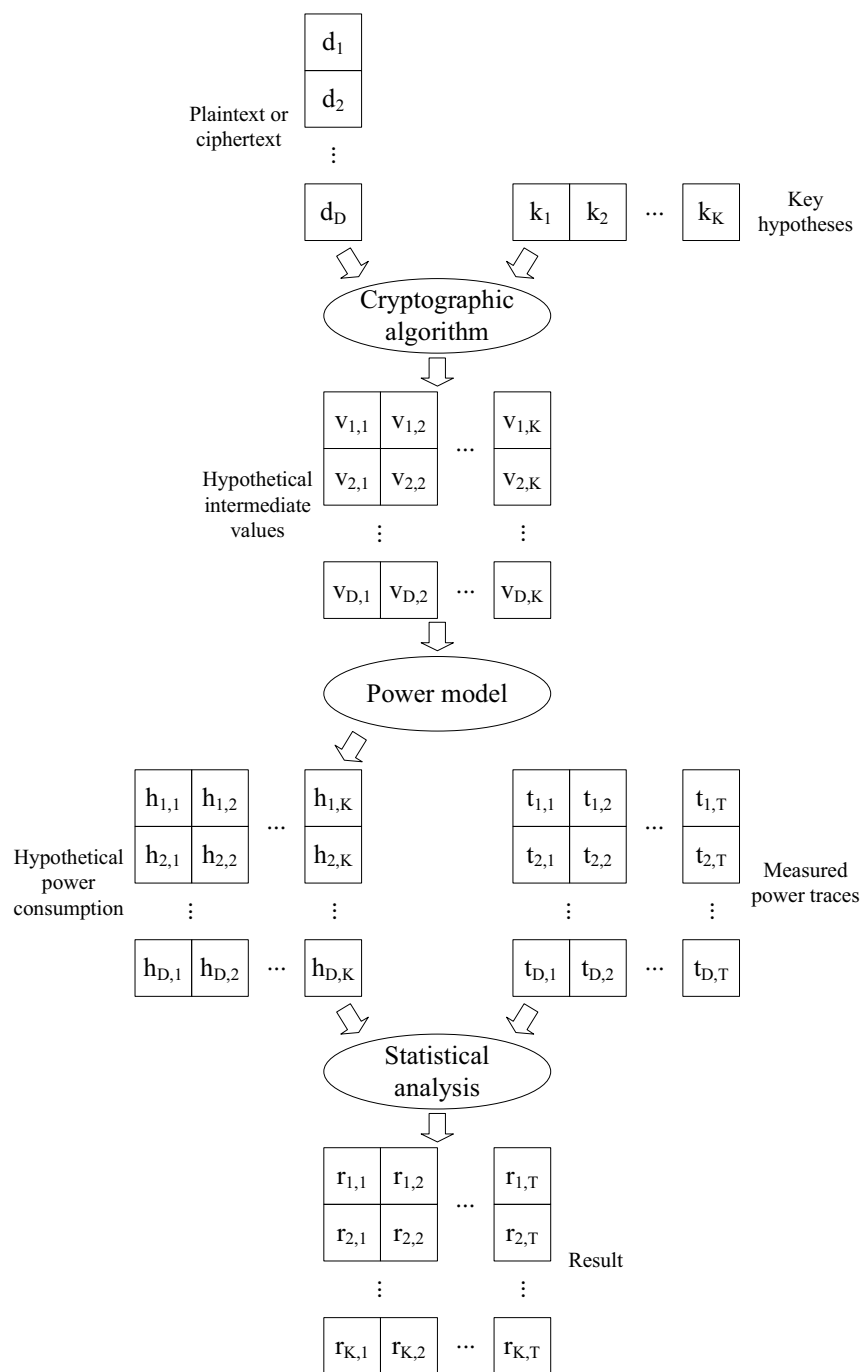


Figure 7: Block diagram illustrating the steps 3 to 5 of a DPA attack.

3 Configuring an attack

We choose the output byte of the first AES S-box in round one (step 1). This intermediate result is a function of the first byte of plaintext and the first byte of the secret key.

Accordingly we sample the power consumption of a simple microcontroller during the first AES round leading to traces corresponding to 1000 different plaintexts. We can organise the collected traces like the matrix $\mathbf{T}$ of power consumption values (step 2).

Next we calculate hypothetical intermediate values based on the 1000 known plaintexts and guess one single byte of the key. This means that we calculate the values $v_{i,j} = S(d_i \oplus k_j)$, where $d_1, \dots, d_{1000}$ are the first byte of each of the 1000 plaintexts and $k_j = j - 1$ with $j = 1, \dots, 256$. The matrix $\mathbf{V}$ has hence a size of 1000×256 values (step 3).

Next we map this matrix to hypothetical power values using the Hamming weight leakage model, leading to a matrix $\mathbf{H}$, $h_{i,j} = HW(v_{i,j})$ (step 4).

Finally, we use the correlation coefficient to determine the linear relationship between the columns $\mathbf{h}_i$ and $\mathbf{t}_j$ for $i = 1, \dots, K$ and $j = 1, \dots, T$ (step 5).

This attack configuration results in a matrix $\mathbf{R}$ of estimated correlation coefficients $r_{i,j}$ as given in (2). In (2), the values $\bar{h}_i$ and $\bar{t}_j$ denote the mean values of the columns $\mathbf{h}_i$ and $\mathbf{t}_j$.

$$r_{i,j} = \frac{\sum_{d=1}^D (h_{d,i} - \bar{h}_i) \cdot (t_{d,j} - \bar{t}_j)}{\sqrt{\sum_{d=1}^D (h_{d,i} - \bar{h}_i)^2 \cdot \sum_{d=1}^D (t_{d,j} - \bar{t}_j)^2}} \quad (2)$$

Figure 8 shows the result of this attack. The plot for key guess 225 is plotted in black colour: it contains the highest peaks, indicating that the correct secret key byte was in fact 225.

We can observe that, using 1000 traces, we have an incredibly strong indicator for the correct key byte, as the correlation values are close to ± 1 . One can wonder if the correct key could hence be identified using fewer traces? To make some way towards answering such a question we could consider repeating the same experiment, using the same trace set, but taking fewer traces to do the correlation estimation. In particular, we could fix the time point at which the correct key has its highest correlation peak, I call this time point t_{ck} , and then, just for this time point do the same attack but always increasing the number of traces. Figure 9 shows the “evolution” of the correlation coefficient for the correct key guess (in black colour) and all the incorrect key guesses (in grey colour) when increasing the number of traces for the correlation estimation. The figure shows very clearly that after just a few traces, the black line is clearly separated from all the grey lines: with just a few traces, there is a good chance that our attack would already

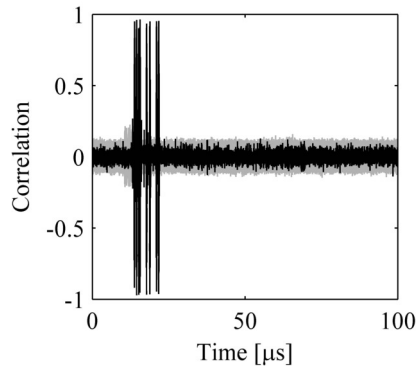


Figure 8: All rows of $\mathbf{R}$. Key hypothesis 225 is plotted in black, while all other key hypotheses are plotted in gray.

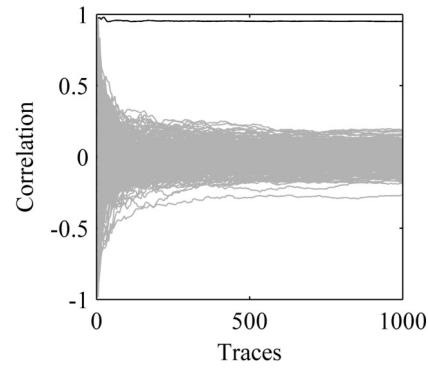


Figure 9: The column of $\mathbf{R}$ at $13.8\mu\text{s}$ for different numbers of traces. Key hypothesis 225 is plotted in black.

succeed.

A mathematically defensible estimation of the “number of needed traces” is obviously a relevant question for practice: not only to “judge” how expensive is a given attack configuration, but also how might some countermeasures change this number.

References

- [MOP07] Stefan Mangard, Elisabeth Oswald, and Thomas Popp. *Power analysis attacks: Revealing the secrets of smart cards*. Springer, 2007.
- [Nat23] National Institute of Standards and Technology. *Advanced Encryption Standard (AES)*. Tech. rep. Federal Information Processing Standards Publication (FIPS) NIST FIPS 197-upd1. Washington, D.C.: Department of Commerce, updated May 9, 2023. URL: <https://doi.org/10.6028/NIST.FIPS.197-upd1>.